Sebenta

Aula 06: BLE Avançado, Gateway, Cliente e Notificações

Tecnologias de Comunicação para a Internet das Coisas

Ano letivo 2026/2027

Catarina Silva

ESTGA, Universidade de Aveiro

Texto de apoio ao estudo

Índice

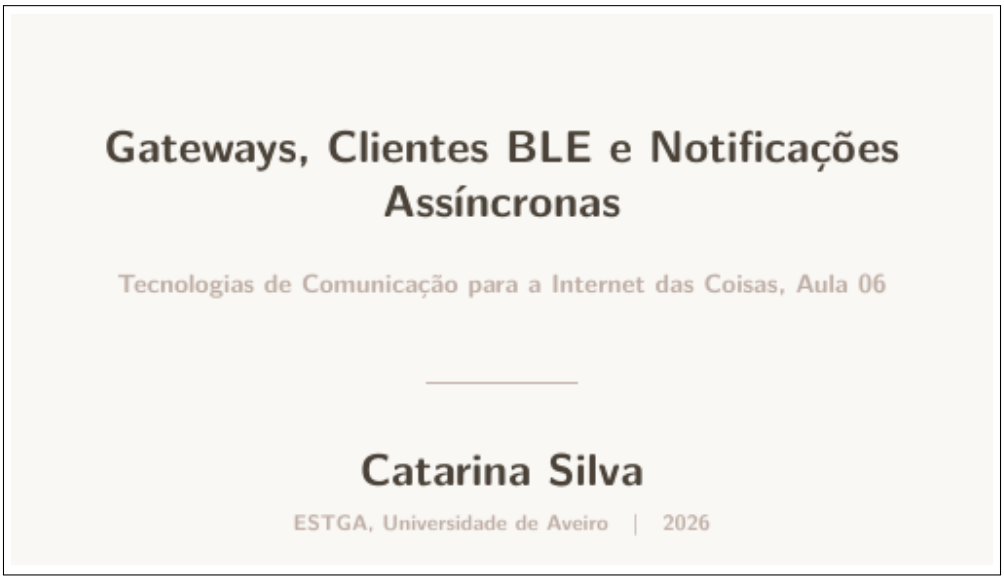
1 Texto por Diapositivo **2**

1. Título . . . 2
2. Agenda . . . 2
3. O gateway entre duas redes . . . 3
4. Arquitetura em camadas . . . 4
5. Papéis e topologia em BLE . . . 5
6. A pilha BLE e onde corre cada camada . . . 6
7. Anatomia de uma sessão BLE . . . 7
8. O modelo de dados GATT . . . 8
9. O Raspberry Pi como cliente BLE: classes da bluepy . . . 9
10. Sondagem e notificação . . . 10
11. De onde vem a latência . . . 11
12. Ritmo da ligação e compromissos . . . 12
13. Débito e fragmentação . . . 13
14. Perda de ligação e recuperação . . . 14
15. Vários nós, um gateway . . . 15
16. Processamento local e memória . . . 16
17. Encaminhamento para a rede IP . . . 17
18. Questões de rede do gateway . . . 18
19. Administração remota . . . 19
20. Segurança e privacidade . . . 20
21. Requisitos de arquitetura e decisões . . . 21
22. Síntese da aula . . . 22
23. Fontes . . . 22
24. Encerramento . . . 23

1. Texto por Diapositivo

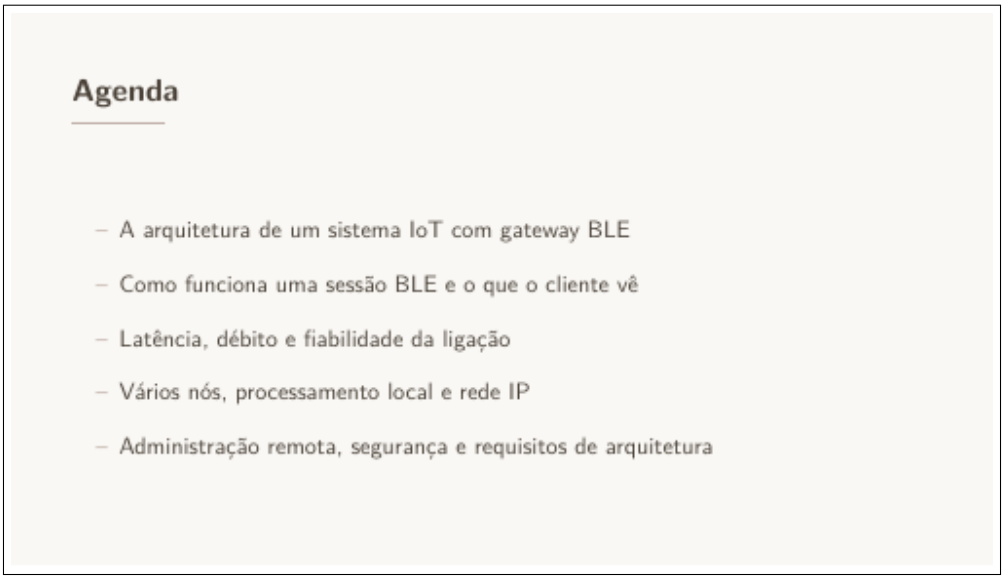
Cada secção corresponde a um diapositivo da aula e desenvolve a matéria que ele introduz. Lidos em sequência, os textos formam uma exposição contínua do tema.

1. Título



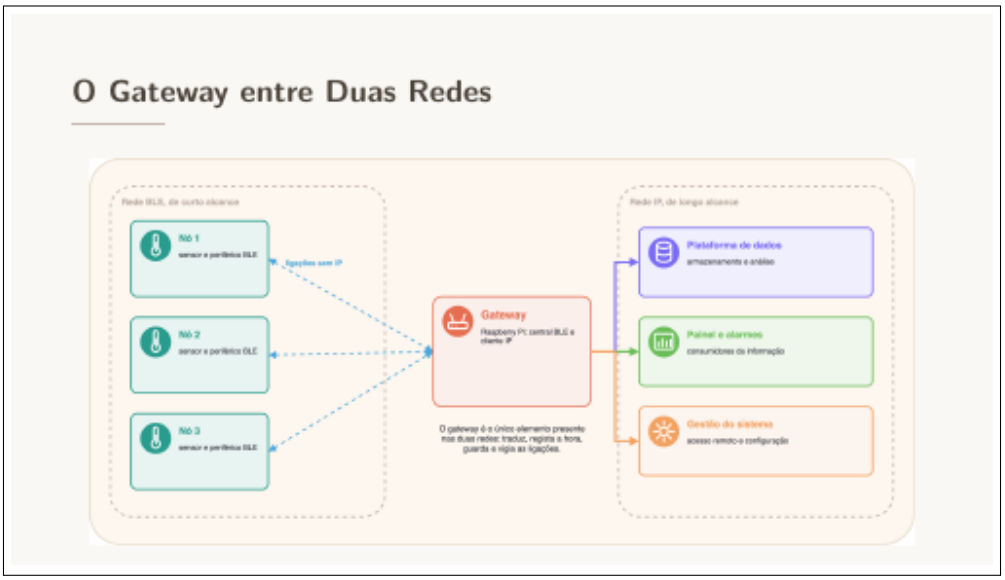
Esta aula é a terceira dedicada ao Bluetooth Low Energy e muda o ponto de vista. Nas sessões anteriores o interesse esteve no dispositivo que se anuncia e responde. Agora o interesse está no elemento que o procura, se liga a ele, recolhe os dados e os entrega a uma rede de dados. Esse elemento chama-se gateway, e a sua conceção levanta questões de arquitetura, de rede, de latência, de escala e de segurança que o texto seguinte percorre.

2. Agenda



O percurso começa pelo gateway enquanto elemento de arquitetura e pelo lugar que ocupa entre as redes. Segue com os mecanismos do BLE que um gateway tem de dominar: papéis, camadas, sessão e modelo de dados. Depois trata o Raspberry Pi como cliente BLE, a diferença entre sondagem e notificação e as causas da latência. A terceira parte aborda o que acontece quando o sistema cresce ou falha: vários nós, perda de ligação e processamento local. Termina com o encaminhamento para a rede IP, a administração remota e a segurança, e com uma tradução dos requisitos em decisões.

3. O gateway entre duas redes



O BLE foi desenhado para ligações curtas entre dispositivos restritos. Um periférico BLE não tem endereço IP, não participa em nenhuma rede de dados e só é alcançável por um dispositivo que esteja ao alcance do seu rádio. A Internet e as redes locais de dados assentam em IP. Entre as duas existe um fosso que nenhum sensor consegue atravessar sozinho.

O gateway atravessa esse fosso porque possui duas interfaces. Uma é um adaptador BLE, através do qual atua como central e mantém ligações com os nós. A outra é uma interface IP, através da qual entrega a informação a quem a consome. Esta posição dá-lhe um conjunto de funções que os nós não conseguem assumir.

A primeira função é a tradução. O gateway recebe valores em notificações do protocolo de atributos e reenvia-os num protocolo da rede IP, com a estrutura que o consumidor espera. A segunda é o registo temporal. Os nós restritos raramente têm relógio fiável, de modo que a hora associada a cada leitura é a da receção no gateway. A terceira é a memória: o gateway dispõe de alimentação permanente e de armazenamento, e por isso pode guardar leituras enquanto a rede IP está indisponível. A quarta é a supervisão das ligações, que inclui detetar a perda de um nó e restabelecer o contacto.

Há também consequências menos confortáveis. O gateway é um ponto único de falha para todos os nós que serve, e é a fronteira de segurança entre duas redes de natureza muito diferente. O seu alcance rádio define a área coberta, o que obriga a pensar na sua localização física.

4. Arquitetura em camadas

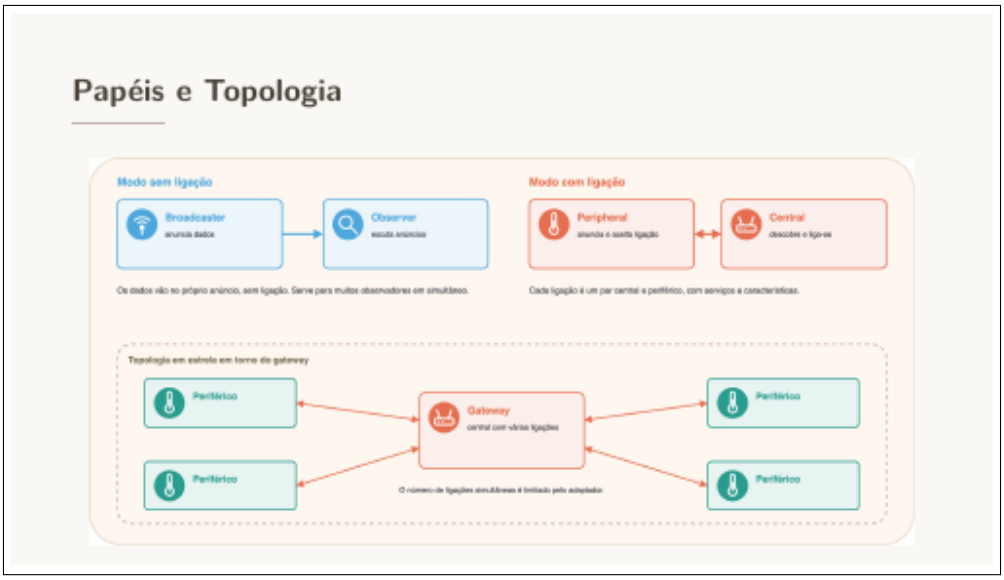


O modelo de referência de IoT usado na Aula 01 organiza um sistema em níveis, dos dispositivos até às aplicações. O diagrama mostra uma versão reduzida com cinco camadas. Na base estão os dispositivos BLE, com sensores e atuadores restritos. Acima está a camada de conectividade e de processamento local, onde vive o gateway. Segue-se a rede IP, que pode ser local, empresarial ou a Internet. Depois vem o armazenamento e o middleware, onde se encontram bases de dados e brokers de mensagens. No topo estão as aplicações, que apresentam, analisam e decidem.

O gateway ocupa um lugar particular. Pertence à camada de conectividade, porque faz a ponte entre tecnologias, mas executa também processamento local, porque filtra, agrega e guarda antes de enviar. Esta dupla natureza explica porque os requisitos de um sistema IoT acabam concentrados nele.

A coluna da direita do diagrama enumera as cinco questões que a restante aula desenvolve. A latência pergunta quanto tempo demora um dado a chegar ao consumidor. A escala pergunta quantos nós um gateway suporta. A fiabilidade pergunta o que acontece quando uma ligação falha. A segurança pergunta quem pode ligar-se e quem pode ler. A administração pergunta como se opera um equipamento sem ecrã nem teclado. Cada uma destas questões tem uma resposta técnica concreta nos mecanismos do BLE e nas escolhas de programação do gateway.

5. Papéis e topologia em BLE



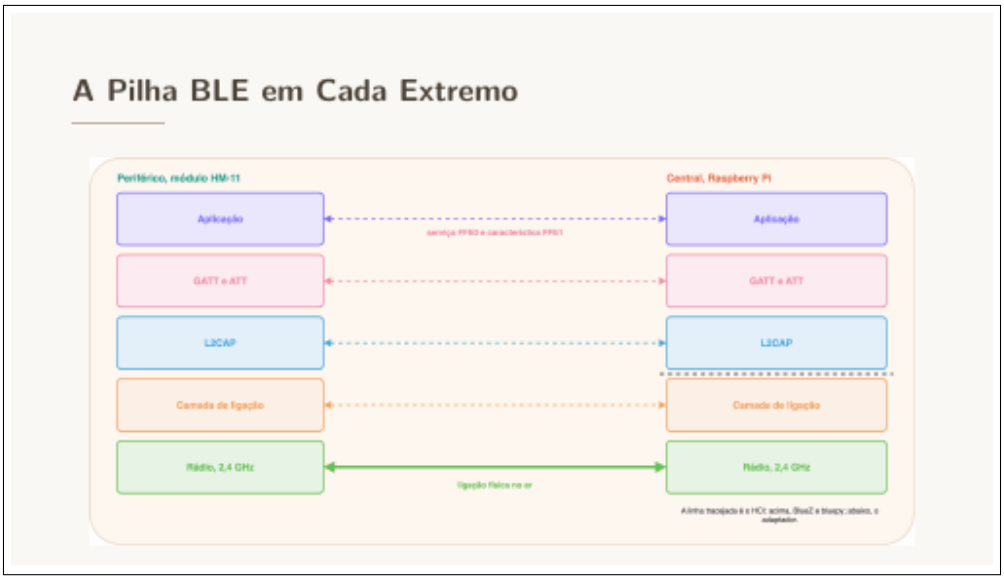
O perfil de acesso genérico, GAP, define os papéis que um dispositivo pode ter. Há dois modos de funcionamento.

No modo sem ligação existem o broadcaster e o observer. O broadcaster emite pacotes de anúncio com dados e não aceita ligações. O observer escuta esses anúncios e retira deles a informação. É um modo útil quando o mesmo dado interessa a muitos recetores e não é preciso uma conversa bidirecional. A capacidade é limitada, porque os dados têm de caber na carga útil do anúncio, e a entrega não é confirmada.

No modo com ligação existem o peripheral e o central. O peripheral anuncia-se e aceita um pedido de ligação. O central escuta, escolhe um periférico e estabelece a ligação. A partir daí existe um canal bidirecional, com serviços e características, e com parâmetros negociados. Um gateway que recolhe dados de sensores usa este modo e comporta-se como central.

A topologia resultante é uma estrela em torno do gateway. O central pode manter várias ligações em simultâneo, mas o número é limitado pelo adaptador e pelo controlador, e cada ligação consome tempo de rádio. Em sentido contrário, um periférico como o módulo HM-11 aceita um único central de cada vez. Esta assimetria tem efeitos práticos: enquanto o gateway estiver ligado, o nó deixa de se anunciar, e nenhum outro dispositivo o encontra. Existe ainda uma tecnologia de malha sobre BLE, com outra topologia, mas não faz parte desta aula.

6. A pilha BLE e onde corre cada camada



A pilha BLE divide-se em duas partes separadas por uma interface padronizada, o HCI. Abaixo do HCI está o controlador, que inclui o rádio e a camada de ligação. Acima está o host, que inclui o L2CAP, o protocolo de atributos, o perfil de atributos genérico e o gestor de segurança. No topo está a aplicação.

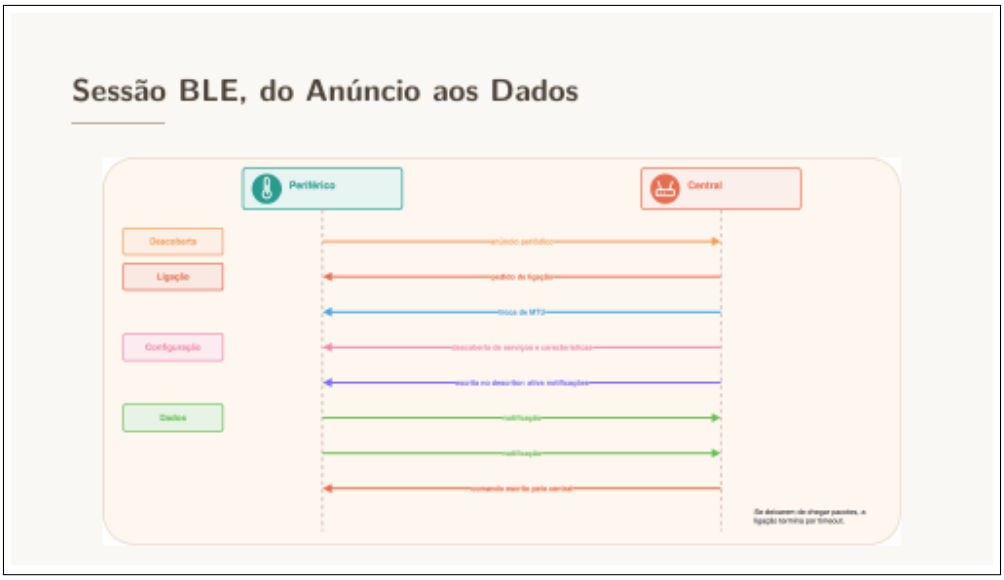
O controlador trata do que é crítico no tempo. Escolhe os canais, aplica o salto de frequência, forma os pacotes, verifica a integridade e retransmite pacotes que não foram reconhecidos. Estas retransmissões ocorrem inteiramente dentro do controlador, e por isso não são visíveis ao nível do HCI.

O host trata do significado dos dados. O protocolo de atributos define o servidor e os seus atributos. O perfil de atributos genérico organiza esses atributos em serviços e características. O gestor de segurança trata de emparelhamento e cifra.

No periférico HM-11 a pilha completa corre dentro do módulo. O Arduino não conhece nada disto: vê apenas uma porta série. Por isso se diz que o módulo expõe uma ligação série transparente.

No Raspberry Pi a divisão é diferente. O controlador está no adaptador Bluetooth da placa, e o host é o BlueZ, que corre no sistema operativo. A biblioteca bluepy acrescenta uma interface em Python por cima do BlueZ. Esta separação importa porque determina o que se pode observar: a ferramenta btmon regista o tráfego no HCI, mostra anúncios, ligações e notificações, mas não revela as retransmissões internas do controlador.

7. Anatomia de uma sessão BLE



Uma sessão completa percorre quatro fases, e compreender a ordem evita muitos erros de programação.

Na fase de descoberta o periférico emite anúncios periódicos e o central escuta. O central identifica o dispositivo pelo endereço, pelo nome ou pelos dados do anúncio.

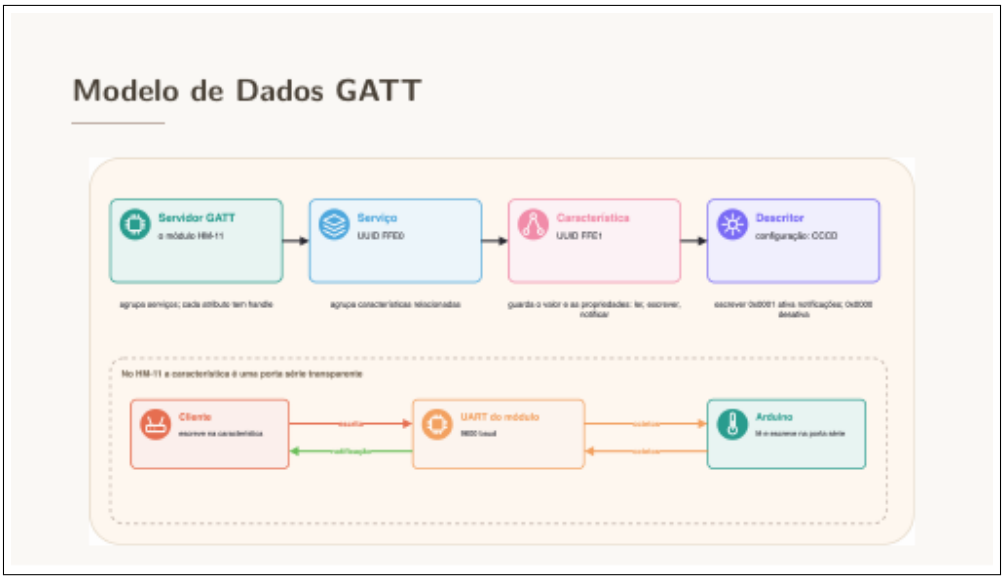
Na fase de ligação o central envia um pedido de ligação que inclui os parâmetros iniciais, como o intervalo entre eventos. A partir desse momento o periférico deixa de se anunciar. Seguidamente, central e periférico trocam o tamanho máximo da unidade de dados do protocolo de atributos, o MTU, que no valor por omissão é de 23 octetos.

Na fase de configuração o central descobre o que o periférico oferece. Pergunta quais são os serviços, depois as características de cada serviço e por fim os descritores. Só depois de conhecer a estrutura sabe a que handle deve escrever. Nesta fase o cliente escreve também no descritor de configuração da característica para ativar as notificações.

Na fase de dados o periférico envia notificações e o central pode escrever comandos. A fase termina quando um dos lados encerra a ligação ou quando o tempo máximo sem pacotes, o supervision timeout, se esgota.

Uma consequência prática é que o descritor de configuração não é ativado automaticamente. Quem se esquece de o escrever liga-se, descobre tudo, e nunca recebe uma notificação. Outra consequência é que as notificações não sobrevivem a uma nova ligação, pelo que um gateway que se religa tem de repetir a fase de configuração.

8. O modelo de dados GATT



O protocolo de atributos define que um servidor expõe uma tabela de atributos. Cada atributo tem um handle, que é um número de 16 bits local ao servidor, um tipo identificado por um UUID e um valor. O perfil de atributos genérico dá estrutura a esta tabela através de uma hierarquia de três níveis.

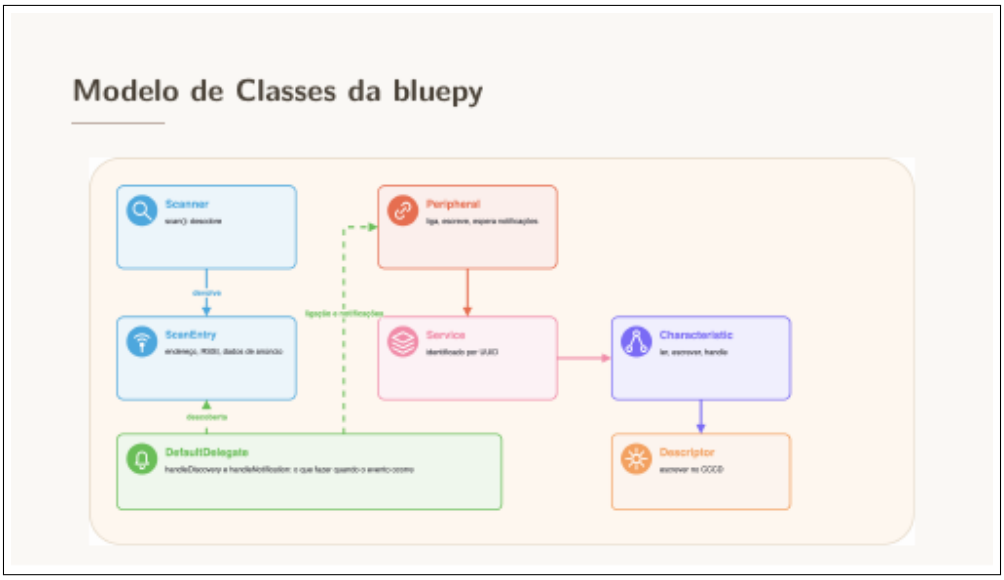
O serviço agrupa características que se relacionam entre si, como as que dizem respeito a um mesmo sensor. A característica contém um valor e um conjunto de propriedades que indicam como pode ser usado: ler, escrever com confirmação, escrever sem confirmação, notificar e indicar. O descritor acrescenta informação sobre a característica, e o mais importante para esta aula é o descritor de configuração, conhecido pela sigla CCCD.

Os UUID podem ter 16 ou 128 bits. Os de 16 bits são abreviaturas de UUID de 128 bits construídos sobre uma base definida pelo Bluetooth. O módulo HM-11 usa o serviço de UUID curto FFE0 e a característica FFE1.

O CCCD tem como UUID o valor 0x2902 e guarda dois bits. O primeiro ativa as notificações e o segundo ativa as indicações. Escrever o valor 0x0001 ativa as notificações e escrever 0x0000 desativa-as. Em muitos servidores simples o descritor ocupa o handle seguinte ao da característica, e é nesse atalho que o código do exercício se apoia. Uma implementação mais robusta descobre os descritores em vez de assumir a posição.

No HM-11 a característica FFE1 funciona como uma porta série transparente, como mostra a parte inferior do diagrama. Os octetos que o cliente escreve na característica saem pela porta série do módulo, a 9600 baud, e chegam ao Arduino. Os octetos que o Arduino escreve na porta série tornam-se o valor da característica e são enviados em notificações. O módulo traduz assim um fluxo de octetos num conjunto de pacotes de atributos.

9. O Raspberry Pi como cliente BLE: classes da bluepy



A biblioteca bluepy dá acesso ao BlueZ em Python e organiza as operações em poucas classes, que correspondem às fases da sessão.

A classe Scanner trata da descoberta. O método de varrimento recebe uma duração e devolve uma lista de entradas. Cada entrada, da classe ScanEntry, contém o endereço, o tipo de endereço, o RSSI e os dados de anúncio, dos quais se pode extrair, por exemplo, o nome do dispositivo.

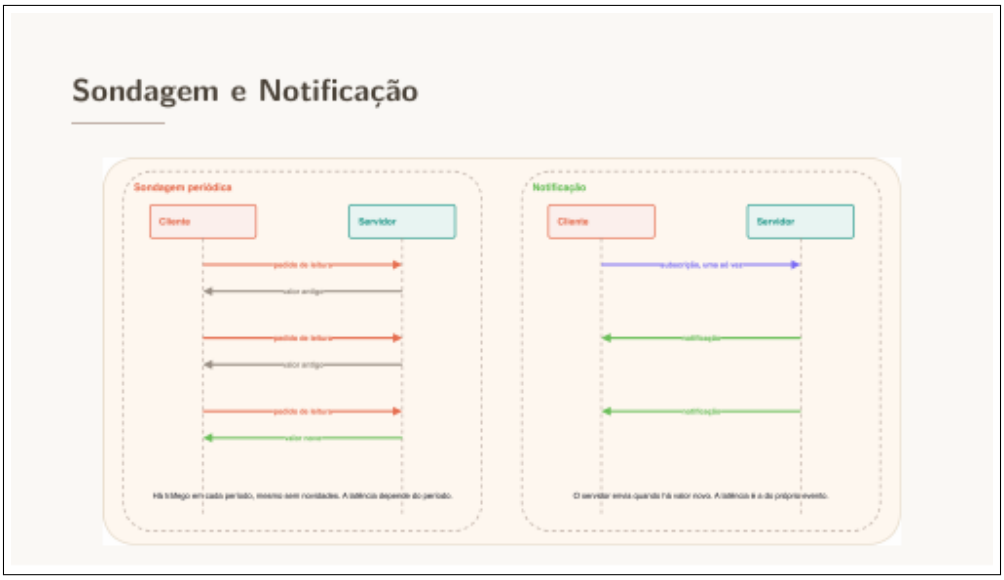
A classe Peripheral representa uma ligação a um dispositivo. O construtor recebe o endereço e bloqueia até a ligação estar estabelecida ou falhar. A partir do objeto obtêm-se os serviços e as características, e escreve-se nos handles, por exemplo para ativar as notificações. O método de espera por notificações bloqueia até chegar uma notificação ou até o tempo acabar.

A classe DefaultDelegate trata dos eventos. O programador define uma subclasse e implementa o método que a biblioteca chama quando acontece algo: a descoberta de um dispositivo ou a chegada de uma notificação. Este padrão chama-se delegação e inverte o controlo. A aplicação não decide quando o evento ocorre, apenas declara o que fazer quando ocorrer.

A inversão de controlo é um padrão central em IoT. Reaparece nos clientes de brokers de mensagens, em que a aplicação regista uma função que é chamada quando chega uma mensagem de um tópico subscrito.

Há duas limitações que condicionam o desenho de um gateway. A biblioteca é síncrona: enquanto espera por notificações, o programa não faz mais nada na mesma linha de execução. E a perda de ligação surge sob a forma de uma exceção, que o programa tem de tratar. Estas duas características voltam a aparecer ao tratar de vários nós e de recuperação de falhas.

10. Sondagem e notificação



Um cliente pode obter dados de uma característica de duas maneiras.

Na sondagem, o cliente envia um pedido de leitura e o servidor responde com o valor atual. Repete o pedido a intervalos regulares. Cada amostra custa dois pacotes de atributos, e o tráfego existe quer o valor tenha mudado ou não. O servidor responde a todos os pedidos, mesmo os que devolvem um valor que o cliente já conhece. A latência fica limitada pelo período de sondagem, porque uma alteração só é vista no pedido seguinte.

Na notificação, o cliente subscreve uma só vez, escrevendo no CCCD, e o servidor envia um pacote sempre que tem um valor novo. Cada amostra custa um pacote, e só existe tráfego quando há dados. A latência é a do próprio evento, a que acresce apenas a espera pela oportunidade de transmissão.

As notificações não são confirmadas pelo protocolo de atributos. Quando é necessária confirmação ao nível da aplicação existe a indicação, que exige uma resposta do cliente antes de enviar a seguinte, o que reduz o ritmo e acrescenta tráfego. Convém não confundir esta falta de confirmação com falta de fiabilidade. A camada de ligação, abaixo, reconhece cada pacote e retransmite os que se perdem enquanto o supervision timeout o permitir. A notificação pode perder-se apenas quando a ligação inteira falha.

A escolha entre os dois modelos depende do que o servidor consegue fazer. A notificação exige que o servidor produza dados por iniciativa própria, e por isso o firmware do nó tem de ser escrito de modo a enviar quando mede, e não apenas quando é perguntado. Quando a aplicação tolera a perda ocasional de uma amostra, a notificação é quase sempre preferível.

11. De onde vem a latência



A latência de um dado, desde que o sensor o produz até ser apresentado, é a soma de várias etapas, e o diagrama enumera-as por ordem. Compreender a cadeia é o que permite saber onde atuar quando o sistema parece lento.

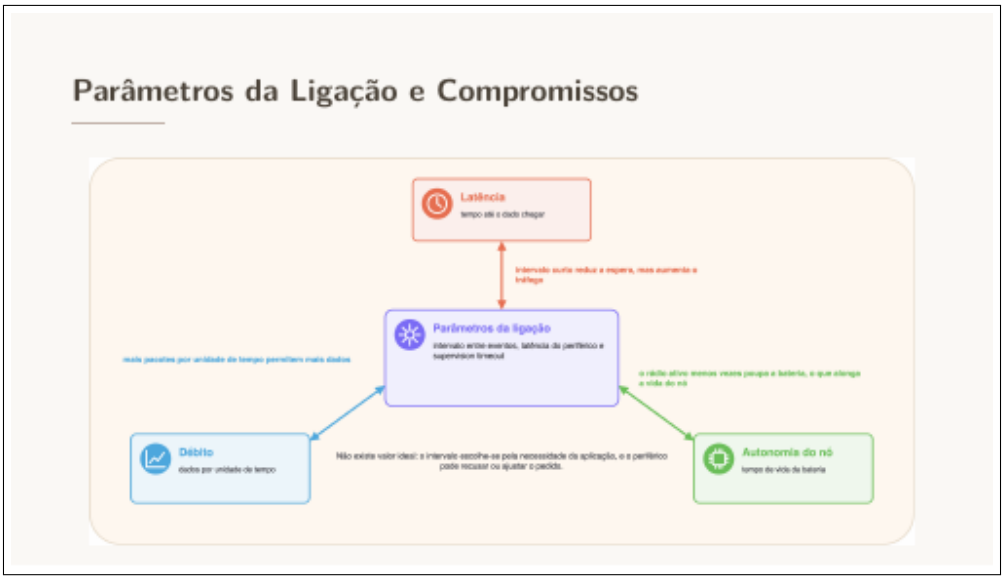
A primeira etapa é a leitura do sensor, que pode demorar o tempo de uma conversão. A segunda é a transmissão pela porta série do módulo, que ocupa o tempo de enviar todos os octetos da mensagem à velocidade configurada. A terceira é a fila de envio na pilha do periférico. A quarta é a espera pelo próximo evento de ligação: o periférico só pode transmitir nos instantes em que o central e ele se encontram, e esse intervalo é um parâmetro da ligação. A quinta é a propagação no ar, que pode incluir retransmissões quando há interferência. Depois vêm o processamento no host, a execução da ponte em Python, a transmissão por TCP e a atualização do painel.

Os traços coloridos sob as etapas mostram quem decide cada parte. O firmware do nó decide as primeiras, os parâmetros da ligação BLE decidem as do meio, e a aplicação do gateway decide as últimas. Só a parte central é determinada pelo protocolo. Em muitos sistemas reais a maior parte da latência está no firmware ou na aplicação, e não no rádio.

Os dois cartões inferiores comparam os modelos de acesso. Com sondagem, o dado espera primeiro pelo próximo pedido do cliente e só depois entra na cadeia. Com notificação, entra na cadeia quando existe e espera apenas o próximo evento de ligação.

É útil distinguir latência de variação da latência e de débito. A latência é o atraso de um dado. A variação da latência, a que se chama jitter, é a diferença entre atrasos de dados consecutivos, e afeta aplicações que dependem de regularidade. O débito é a quantidade de dados por unidade de tempo. Um sistema pode ter bom débito e má latência, e o inverso.

12. Ritmo da ligação e compromissos



Três parâmetros definem o ritmo de uma ligação. O intervalo de ligação é o período entre dois eventos. A latência do periférico é o número de eventos que ele pode ignorar quando não tem nada a enviar. O supervision timeout é o tempo sem pacotes válidos após o qual a ligação é dada como terminada.

O central propõe os valores e o periférico pode pedir alterações, que o central aceita ou recusa. Por isso os valores efetivos podem diferir dos que a documentação do módulo indica.

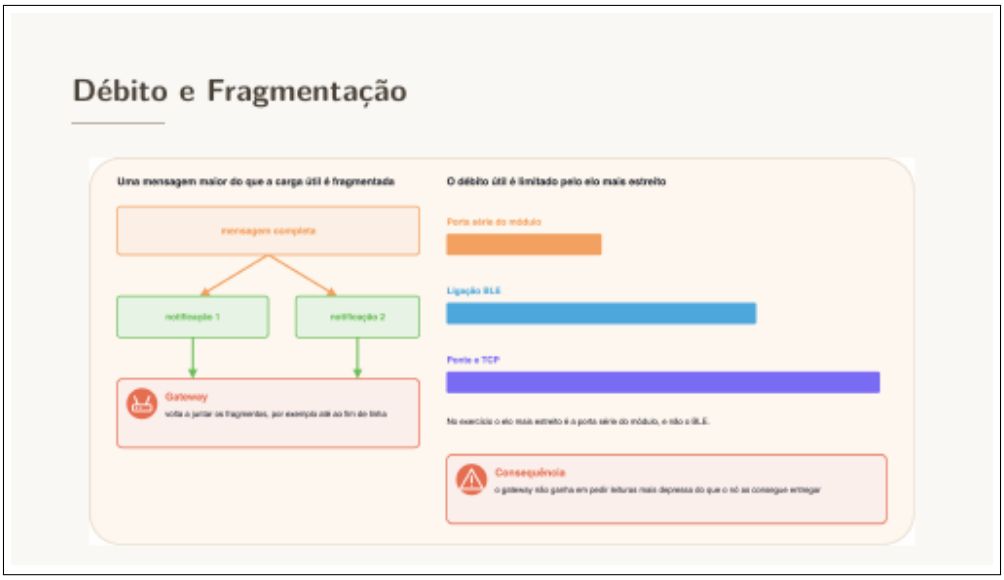
O diagrama mostra que estes parâmetros colocam três propriedades em tensão, e que não existe ajuste que as favoreça todas. Um intervalo curto reduz a espera pelo evento de ligação e permite mais pacotes por unidade de tempo, o que melhora a latência e o débito. Mas obriga o rádio a estar ativo mais vezes, o que reduz a autonomia de um nó alimentado por bateria. Um intervalo longo faz o contrário.

A latência do periférico permite poupar rádio quando o nó tem pouco a dizer. O custo é que um comando enviado pelo central pode esperar vários eventos até ser ouvido, o que pesa em sistemas com atuadores.

O supervision timeout é um compromisso entre rapidez e estabilidade. Um valor curto deteta depressa uma falha, mas pode terminar ligações que apenas atravessaram uma interferência passageira. Um valor longo tolera perturbações, mas demora a revelar uma falha verdadeira. Tem de ser suficientemente maior do que o intervalo, ponderado pela latência do periférico, para que algumas retransmissões caibram dentro dele.

A escolha faz-se a partir do requisito da aplicação. Um sensor de temperatura numa sala tolera intervalos longos. Um comando manual a um atuador exige respostas rápidas.

13. Débito e fragmentação



O MTU por omissão do protocolo de atributos é de 23 octetos. Retirados os 3 octetos do cabeçalho, cada notificação transporta no máximo 20 octetos de dados. Uma mensagem maior é dividida em vários pacotes consecutivos, e o recetor tem de a voltar a montar. Como o BLE não preserva as fronteiras da mensagem de aplicação, é habitual terminar cada mensagem com um delimitador, como o fim de linha, e juntar fragmentos até o encontrar.

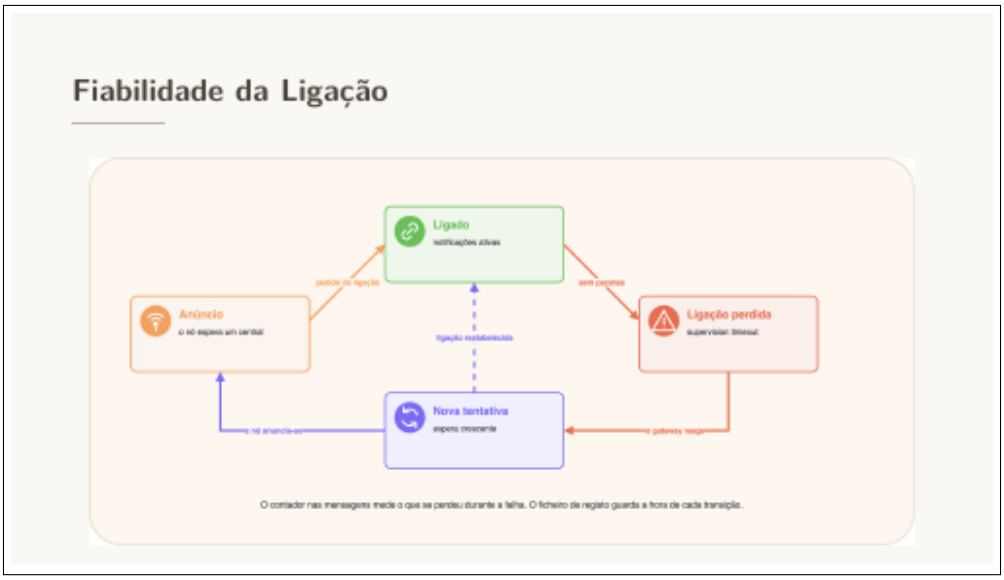
Existem mecanismos para negociar unidades maiores, tanto ao nível do MTU como ao nível do pacote de ligação, mas dependem de ambos os lados os suportarem. Os módulos simples como o HM-11 ficam nos valores por omissão.

O débito útil não é o débito do BLE. É o menor dos débitos de todos os elos por onde os dados passam. O diagrama compara três. A porta série do módulo, a 9600 baud, transfere menos de mil octetos por segundo. A ligação BLE tem capacidade bastante superior. A ponte em Python e a ligação TCP local são ainda mais folgadas.

O elo mais estreito, neste sistema, é a porta série, e não o rádio. A consequência prática é que pedir leituras mais depressa do que o nó as consegue entregar não melhora nada, apenas enche filas. Um gateway bem desenhado conhece o limite do nó e dimensiona o ritmo de acordo com ele.

A estratégia mais eficaz para melhorar o débito de aplicação é reduzir o tamanho das mensagens. Uma mensagem compacta, com poucos campos, cabe num único pacote, evita a fragmentação e reduz o peso relativo dos cabeçalhos. O formato rico, com identificador, unidade e hora, constrói-se no gateway e não é transmitido pelo ar.

14. Perda de ligação e recuperação



Uma ligação BLE pode terminar por vontade de um dos lados ou por perda. Nos sistemas reais a perda é frequente: o nó afasta-se, fica sem bateria, é bloqueado por um obstáculo ou sofre interferência prolongada.

O diagrama descreve os estados que o gateway percorre. Com a ligação estabelecida, as notificações fluem. Quando o periférico deixa de responder, a ligação só é considerada perdida no fim do supervision timeout, de modo que a deteção nunca é imediata. A biblioteca assinala então a perda, no caso da bluepy através de uma exceção, e o programa passa à fase de nova tentativa.

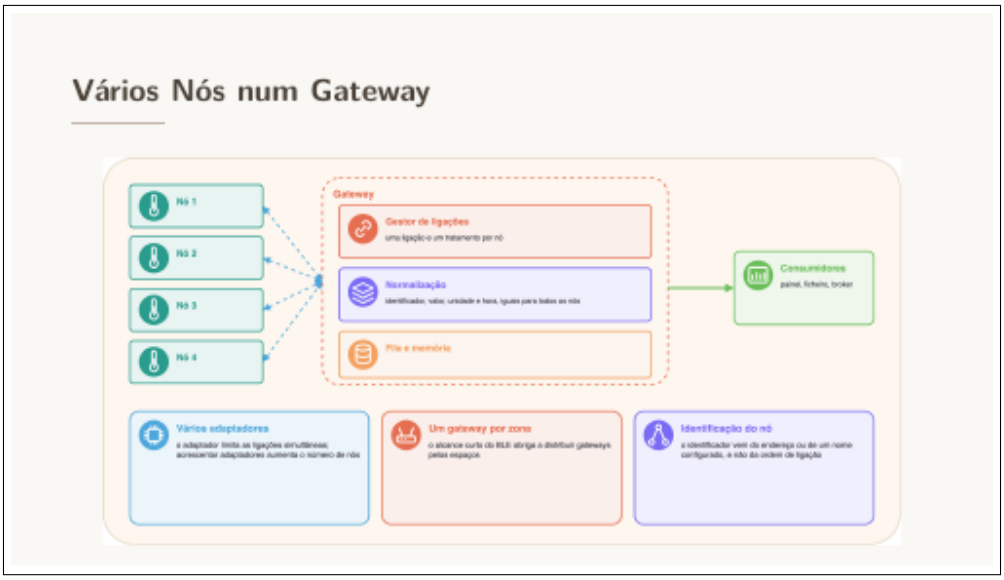
Na nova tentativa, o gateway volta a tentar ligar-se. A estratégia deve evitar tentativas contínuas sem pausa, que saturam o adaptador e impedem outras ligações. O procedimento habitual é aumentar progressivamente o intervalo entre tentativas até um máximo, o que se chama espera crescente. Quando o nó volta a anunciar-se, a ligação restabelece-se.

Restabelecer a ligação não restabelece a configuração. O descritor de configuração volta ao estado inicial, e o gateway tem de repetir a escrita que ativa as notificações. Tem também de retomar qualquer modo de operação que dependa do nó, como o envio espontâneo.

Existem dois problemas que a religação não resolve sozinha. O primeiro é saber quantos dados se perderam durante a falha. Resolve-se numerando as mensagens no nó: uma falha na sequência recebida quantifica as perdas. O segundo é detetar o nó que parece ligado mas deixou de produzir dados. Resolve-se com uma verificação de atividade na aplicação, que declara o nó inativo quando não chegam leituras durante um certo tempo.

O registo da hora de cada transição, perda e restabelecimento, completa a informação necessária para avaliar a fiabilidade do sistema.

15. Vários nós, um gateway



Um gateway real serve muitos nós, e a passagem de um para vários muda o desenho do programa.

A primeira questão é a concorrência. Cada nó tem a sua ligação, os seus parâmetros e o seu fluxo de dados. Como a bluepy é síncrona, um programa com uma só linha de execução só consegue esperar por um nó de cada vez. As soluções habituais são um processo ou uma linha de execução por nó, ou um ciclo de eventos que multiplexa as ligações. Em qualquer dos casos é útil uma unidade de gestão de ligações, representada no diagrama, que cria, supervisiona e fecha cada ligação individualmente.

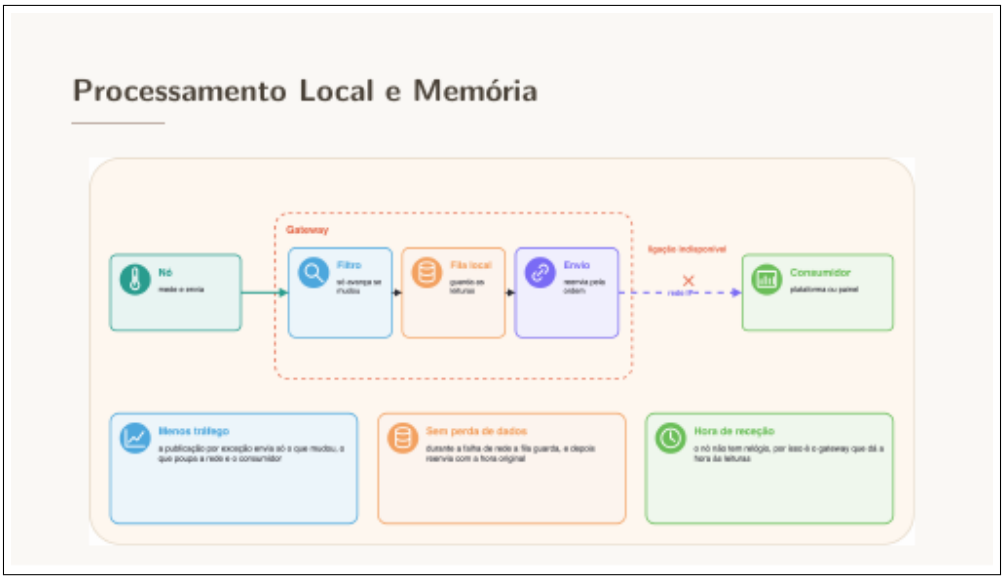
A segunda questão é o limite do adaptador. O controlador tem de intercalar os eventos de ligação de todos os nós sem sobreposição. Quanto mais ligações, e quanto mais curtos os seus intervalos, mais disputado fica o tempo de rádio, e algumas ligações passam a sofrer atraso. O número de ligações simultâneas que um adaptador suporta é finito e depende do controlador. Para ultrapassar o limite acrescentam-se adaptadores ou gateways.

A terceira questão é a identificação. Os nós têm de ser distinguidos entre si de forma estável. O identificador não deve depender da ordem em que as ligações se estabelecem, mas do endereço do dispositivo ou de um nome atribuído por configuração.

A quarta questão é a uniformização dos dados. O gateway converte as mensagens de cada tipo de nó para um formato comum, com identificador, valor, unidade e hora. Esta normalização desacopla os consumidores da diversidade dos nós: um consumidor que lê o formato comum não precisa de saber se o valor veio de um sensor BLE ou de outra tecnologia.

Por fim, o alcance do BLE é curto. Uma instalação distribuída por vários espaços precisa de um gateway por zona, o que levanta o problema da gestão de muitos gateways, da sua configuração e da sua monitorização.

16. Processamento local e memória



Nem tudo o que o gateway recebe deve ser reenviado imediatamente, e nem tudo precisa de ser enviado. O gateway é o lugar natural para decisões locais, porque está perto dos nós e tem recursos.

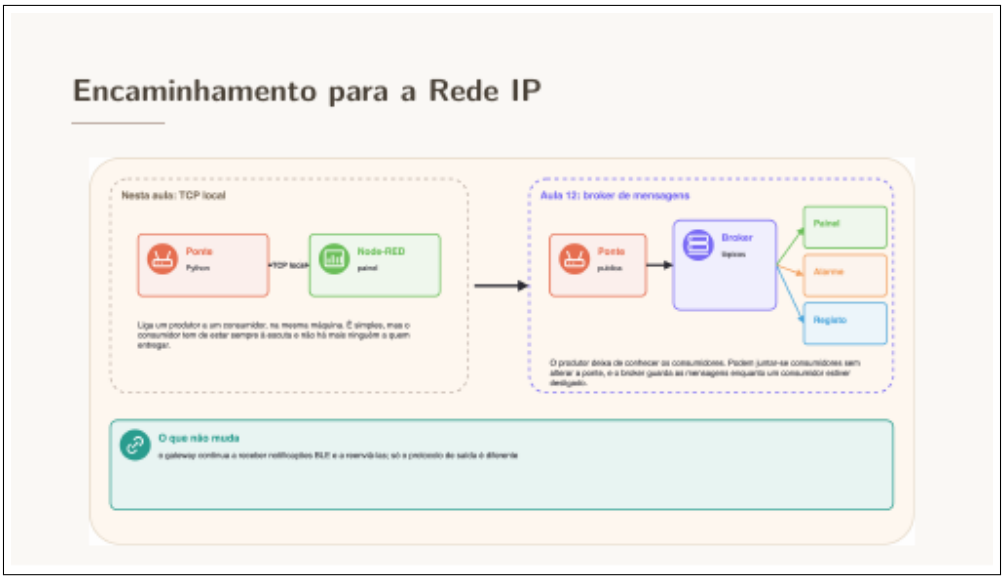
O filtro reduz o tráfego. Na publicação por exceção, o gateway só encaminha uma leitura quando o valor muda mais do que um limiar. Em sistemas que medem grandezas que variam devagar, isto elimina a maior parte das mensagens sem perder informação útil. Reduz a carga na rede, no consumidor e no armazenamento.

A fila local dá resistência a falhas da rede. Quando a ligação à rede IP cai, o gateway não perde as leituras: guarda-as e reenvia-as quando a rede regressa. O padrão chama-se armazenar e reencaminhar. Tem de resolver algumas questões práticas. A fila deve manter a ordem das leituras. Deve ter um tamanho limitado, com uma política clara sobre o que descartar quando enche. Deve poder ser persistente, em disco, se o gateway puder reiniciar. E o reenvio pode produzir duplicados, que o consumidor tem de saber tratar.

A hora tem de ser atribuída na receção e preservada no reenvio. Como o nó não tem relógio, é o gateway que carimba as leituras. Se a hora fosse atribuída apenas no envio, uma leitura guardada durante uma falha de rede pareceria recente, e as análises temporais ficariam erradas.

Este desenho tem uma vantagem adicional. O gateway pode tomar decisões, como disparar um alarme local, mesmo que a rede ou a plataforma remota estejam indisponíveis.

17. Encaminhamento para a rede IP



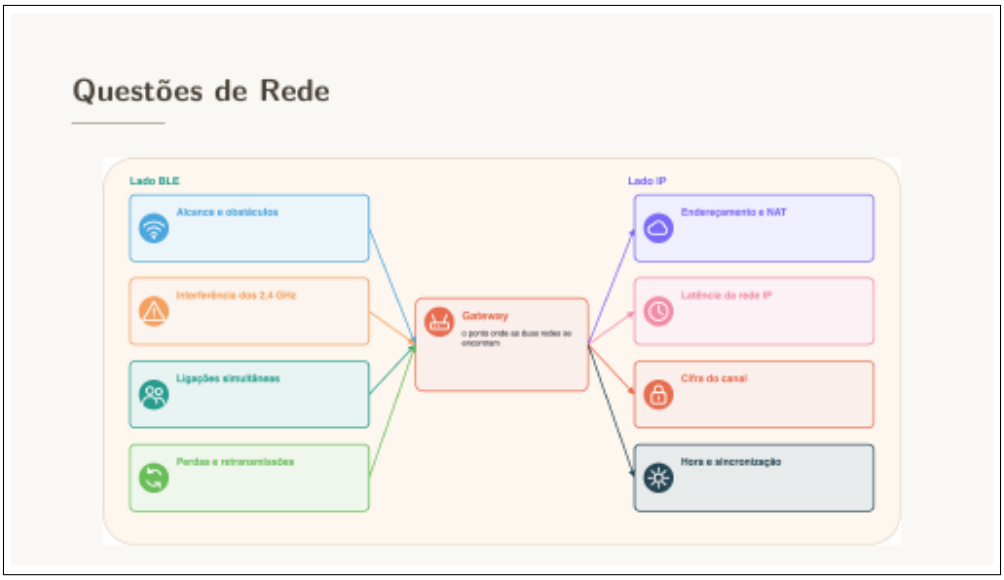
Depois de recebidas e tratadas, as leituras têm de chegar a quem as usa. O diagrama compara duas formas de o fazer.

Na primeira, que é a usada nesta aula, a ponte em Python liga-se por TCP local ao Node-RED. O produtor e o consumidor estão na mesma máquina e comunicam por uma ligação direta. É simples e dispensa infraestrutura. Tem limitações claras. O consumidor tem de estar sempre ativo e à escuta. O produtor conhece o consumidor, e por isso acrescentar um segundo consumidor obriga a alterar a ponte. E não há memória intermédia: se o consumidor estiver parado, os dados perdem-se.

Na segunda, que será estudada na Aula 12, a ponte publica as mensagens num broker. O broker organiza as mensagens por tópicos, e os consumidores subscrevem os tópicos que lhes interessam. O produtor deixa de conhecer os consumidores. É possível acrescentar painéis, alarmes e registos sem tocar na ponte. O broker pode reter mensagens enquanto um consumidor está desligado. Diz-se que produtor e consumidor ficam desacoplados no espaço, porque não se conhecem, e no tempo, porque não têm de estar ativos em simultâneo.

A estrutura do gateway mantém-se nas duas versões. Continua a receber notificações BLE, a normalizá-las e a reenviá-las. Muda apenas o protocolo da interface de saída. É por isso que a arquitetura do gateway se estuda antes dos protocolos de aplicação e do middleware de mensagens: o que se aprende agora aplica-se sem alteração a esses protocolos.

18. Questões de rede do gateway

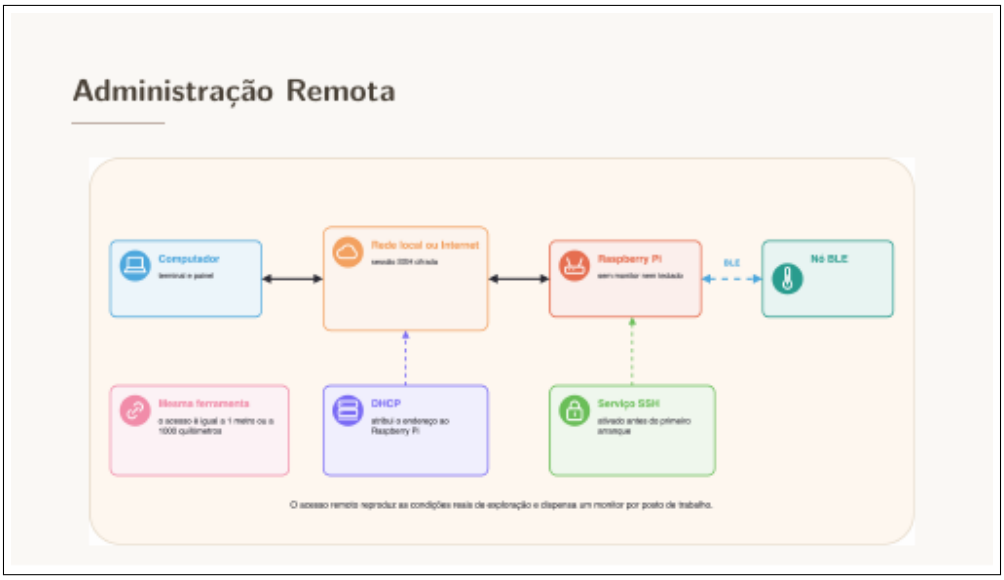


O gateway fica no meio de duas redes com problemas diferentes.

Do lado do BLE, o alcance depende da potência, da antena, dos obstáculos e da orientação. As paredes, os metais e os corpos humanos atenuam o sinal e fazem variar o RSSI, que por isso não serve de medida fiável de distância. A banda dos 2,4 GHz é partilhada com o Wi-Fi e com outros equipamentos. O salto de frequência adaptativo do BLE evita os canais ocupados, mas a interferência continua a causar retransmissões e a degradar o ritmo. Num Raspberry Pi há ainda uma particularidade: o rádio Wi-Fi e o Bluetooth partilham a mesma placa e podem competir pelo mesmo espetro. O número de ligações simultâneas é limitado, e a taxa de perdas e retransmissões condiciona a capacidade efetiva.

Do lado IP, o endereçamento merece atenção. Um gateway atrás de um equipamento de tradução de endereços consegue iniciar ligações para fora, mas não recebe ligações de fora sem configuração adicional. O endereço do próprio gateway pode mudar se for atribuído por DHCP. A latência e as perdas da rede de longa distância podem ser muito maiores do que as da rede local, e o desenho deve tolerá-las, o que reforça o papel da fila local. O canal para a rede tem de ser cifrado, porque os dados atravessam redes que não controlamos. E a hora do gateway tem de ser correta, porque é ele que carimba as leituras. Um relógio errado corrompe todos os dados e deve ser sincronizado com um serviço de tempo da rede.

19. Administração remota



Um gateway instalado no terreno raramente tem ecrã e teclado. Toda a configuração, diagnóstico e atualização faz-se à distância, e este modo de operação diz-se sem cabeça.

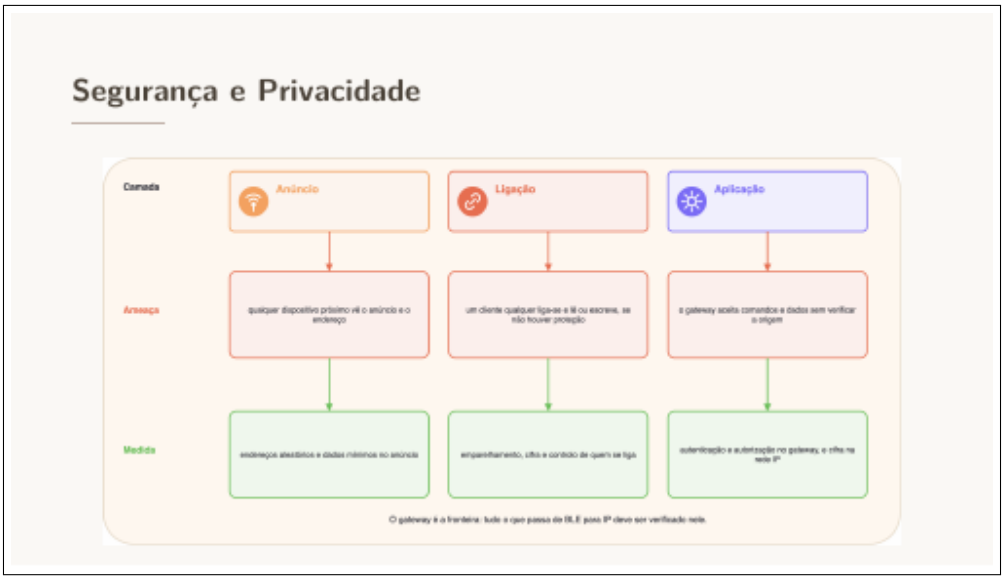
O instrumento habitual é o SSH, que abre uma sessão de linha de comandos cifrada sobre a rede. A sessão funciona da mesma maneira para um gateway na mesma sala ou a milhares de quilómetros, desde que haja um caminho de rede até ele.

Para chegar ao gateway é preciso saber o seu endereço. Numa rede local, o servidor DHCP atribui-o, e descobre-se consultando a lista de clientes do servidor, pelo nome da máquina ou por um varrimento da rede. Uma instalação duradoura beneficia de um endereço reservado ou de um nome estável.

O serviço SSH tem de estar ativo. No Raspberry Pi, nas versões usadas na UC, ativa-se criando um ficheiro vazio com o nome ssh na partição de arranque do cartão antes do primeiro arranque. Noutras versões, a opção é escolhida durante a preparação do cartão.

Alguns cuidados completam a prática. Para um gateway acessível fora da rede local, a autenticação por chave é mais segura do que a palavra passe, e um acesso por rede privada virtual é preferível à abertura direta de portas. Convém verificar remotamente o estado do adaptador Bluetooth, que é o primeiro ponto de falha de um gateway BLE. E manter um registo dos eventos do gateway permite diagnosticar à distância o que correu mal.

20. Segurança e privacidade



A segurança de um sistema BLE com gateway decide-se em camadas, e o diagrama relaciona cada uma com a sua ameaça e a sua medida.

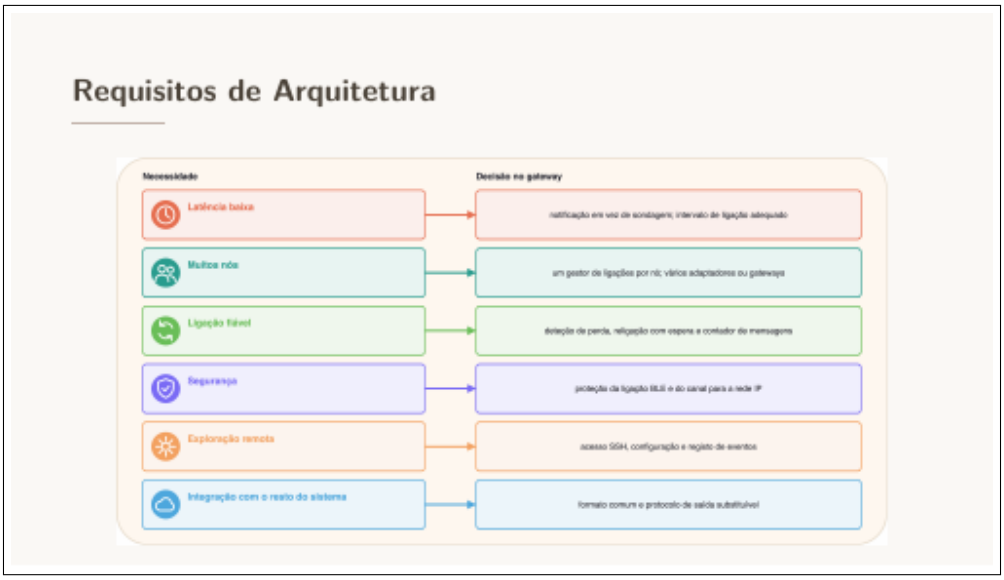
Na camada de anúncio, qualquer dispositivo próximo vê os pacotes e o endereço. Isto permite a recolha de informação sobre os dispositivos de um espaço sem qualquer autorização, e permite seguir uma pessoa através de um dispositivo que use sempre o mesmo endereço. A medida de privacidade é usar endereços aleatórios que mudam ao longo do tempo e manter os dados do anúncio no mínimo. Os endereços podem ser públicos, atribuídos pelo fabricante, ou aleatórios, e só os aleatórios renováveis protegem contra o seguimento.

Na camada de ligação, a ameaça é que um cliente qualquer se ligue, leia e escreva. Um módulo como o HM-11, na configuração por omissão, aceita qualquer central e não cifra o tráfego. A medida é usar emparelhamento e cifra da ligação, e limitar quem se pode ligar. O BLE dispõe de mecanismos de emparelhamento e ligação segura que o módulo pode ou não suportar, e esse suporte deve ser verificado antes da escolha do módulo.

Na camada de aplicação, o gateway não deve aceitar dados ou comandos sem verificar a origem. Deve validar o formato do que recebe, limitar o ritmo dos comandos e autenticar quem lhe dá ordens. Para a rede IP, o canal deve ser cifrado, e as credenciais de acesso não devem estar escritas no código, mas em variáveis de ambiente ou em ficheiros protegidos.

O gateway é a fronteira entre duas redes e o seu ponto de controlo. O que passa de BLE para IP deve ser verificado nele, e o que desce da rede IP para os nós deve ser autorizado nele.

21. Requisitos de arquitetura e decisões

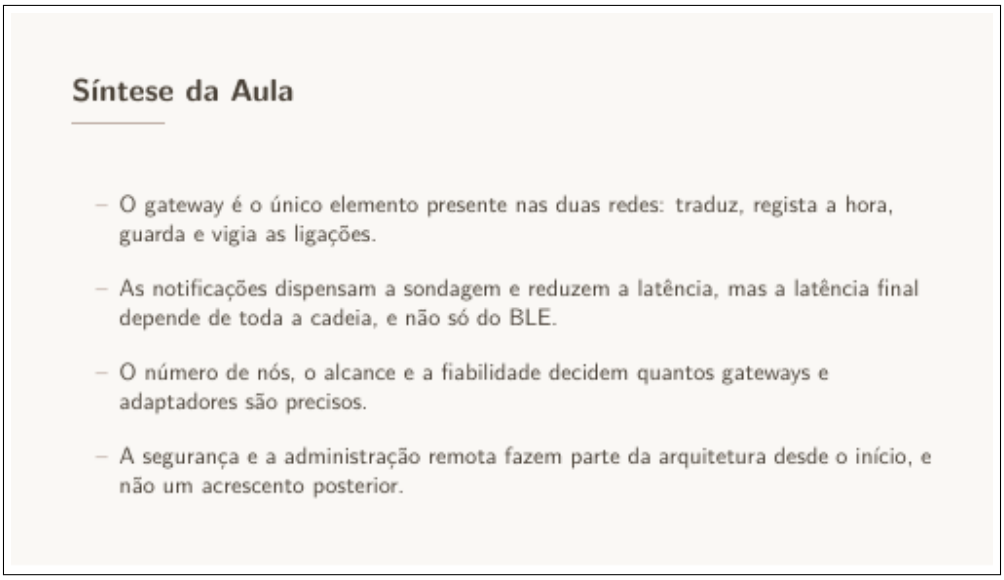


O último diagrama resume a aula como uma tabela de correspondência entre necessidades e decisões. Cada linha começa por uma necessidade do sistema e termina numa decisão tomada no gateway.

Para latência baixa, a decisão é usar notificações em vez de sondagem e escolher um intervalo de ligação adequado ao requisito. Para suportar muitos nós, a decisão é dotar o gateway de uma unidade de gestão de ligações com tratamento independente por nó, e recorrer a vários adaptadores ou gateways quando o limite for atingido. Para fiabilidade, a decisão é detetar a perda, religar com espera crescente, repetir a configuração e numerar as mensagens para medir as perdas. Para segurança, a decisão é proteger a ligação BLE e o canal para a rede IP e verificar na fronteira o que passa. Para exploração remota, a decisão é aceder por SSH, configurar à distância e registar eventos. Para integração com o resto do sistema, a decisão é adotar um formato comum e manter o protocolo de saída substituível.

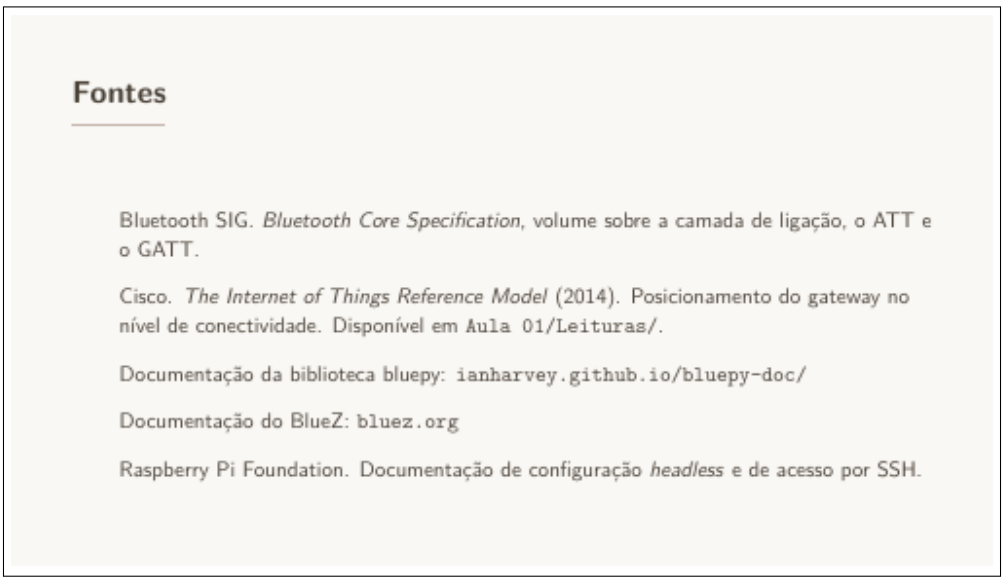
A ideia que atravessa as seis linhas é que as qualidades de um sistema IoT resultam de decisões de arquitetura, e não apenas da escolha da tecnologia de rádio. O BLE fornece os mecanismos, mas é o desenho do gateway que decide se o sistema é rápido, escalável, fiável e seguro.

22. Síntese da aula



O gateway liga a rede BLE, sem IP, à rede de dados, e concentra tradução, memória, hora e supervisão. A notificação reduz o tráfego e a latência face à sondagem, mas a latência final é a soma de uma cadeia de etapas, das quais só parte pertence ao BLE. A escala, a fiabilidade e a segurança são requisitos de arquitetura que se resolvem no gateway, através da gestão de várias ligações, da religação com espera crescente e da verificação na fronteira. O encaminhamento por TCP local será substituído por um broker de mensagens, sem alterar a estrutura do gateway. Estes temas voltam nas aulas seguintes, com outras tecnologias de acesso e com os protocolos de aplicação.

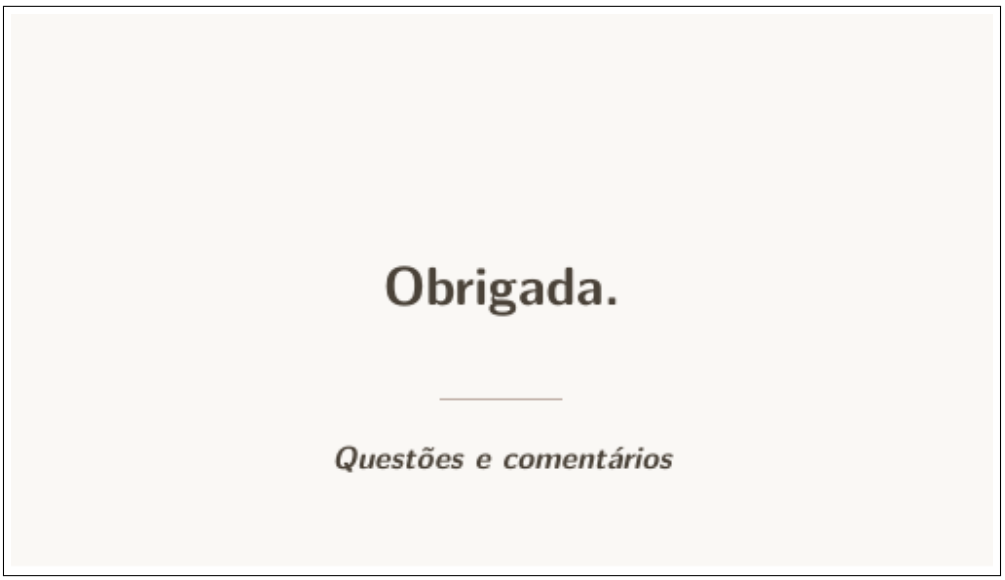
23. Fontes



A especificação central do Bluetooth é a referência para os papéis, o protocolo de atributos, o perfil de atributos genérico e a camada de ligação, e é onde se confirmam os intervalos permitidos

para cada parâmetro. O modelo de referência de IoT da Cisco, disponível nas leituras da Aula 01, enquadra o lugar do gateway na arquitetura. A documentação da biblioteca bluepy descreve as classes e métodos usados pelos programas da UC, e a do BlueZ descreve a pilha que corre no Raspberry Pi. A documentação da Raspberry Pi Foundation cobre a configuração sem cabeça e o acesso por SSH. Os guiões de edições anteriores serviram de base à parte de administração remota e de cliente BLE.

24. Encerramento



O essencial a reter é que um gateway não é apenas um adaptador entre tecnologias. É o ponto onde se decidem latência, escala, fiabilidade, segurança e operação, e o seu desenho condiciona tudo o que está acima dele.